

B's build sheet, decision layer + UI

Full reasoning lives in `BUILD-PLAN.md` and `docs/` in the repo. This page is just what you do, in order.

You are not blocked by Rain. Start now.

First, before any checker code: pick storage

In-memory or on-disk state does not survive a Vercel cold start. The append-only decision log needs somewhere real to live, or replay works on localhost and silently breaks on the exact URL you're required to submit.

Pick Neon Postgres (free tier, fits the append-only model directly, queryable for replay) **or Vercel KV / Upstash** (faster to wire, less natural for the replay query). Five minutes, decide, move on. This blocks everything else you build today.

Your three-hour blocks

13:00-15:00: the six checks, pure functions, tests, no UI

Agree the PO shape with A first: `{ poNumber, vendor, sku, unitPrice, quantity, quoteExpiry }`.

Each check takes the PO plus the record it's checked against, returns pass/fail plus a plain-English reason, never "validation failed."

1. PO exists and is accepted
2. PO still open, not already fulfilled
3. Amount matches the quote
4. Merchant matches the quote (*secondary, see note below*)
5. Within remaining budget
6. **No card already issued for this PO** (*idempotency. This one carries the demo. Get it right first.*)

Rules live as versioned config data, not as hardcoded logic. A rule is a row: id, type, parameters, enabled, version. Editing one creates a new version, the old one is never mutated. This is what makes replay possible at all.

Write `verify(po, record, rules) -> { ok, failures[] }` as a pure function with zero I/O. Test it against fixtures before touching a database or a UI.

 On check 4, don't claim "every Rain control passes this" until A confirms with the engineer whether Rain locks to category or exact merchant. If exact, reframe: "we check the same thing Rain does, plus the parts of the deal Rain never sees."

15:00-17:00: storage wired, decision log, UI shell, seed data

- Wire the storage you picked. **Store a snapshot of the record at decision time, not a pointer to it.** Records change; replaying a pointer six hours later re-judges today's facts, not the facts that were actually true then.
- The decision log is append-only: every evaluation writes one row, PO, record snapshot, rule version, per-check result, outcome. Nothing updated in place, nothing deleted.
- Build the UI shell: the transcript/decision feed, a provenance panel (four fields on any refusal: rule that failed, expected value, actual value, record it read), and the rule editor.
- **Seed 40-50 historical decisions now.** Six live decisions makes replay read like a test file. Forty-plus makes the diff read like a real system. Half an hour, do it before 17:00.

17:00: the join

A's agent produces a real PO. Your `verify()` runs against it for real. Pass, A calls `issueRainCard()`. Fail, no card, your reason is what displays.

If this isn't working by 18:00, stop and cut scope, starting with the negotiation stage, not with your checks or storage.

18:00-20:00: replay, the provenance panel, the run-it-twice UI

- **Replay:** edit a rule in the UI, it saves as a new version, re-run every seeded decision against it, show the diff. *"Across 47 decisions, 6 approvals would now be refused, 2 refusals would now pass."* This is the headline feature.
- Make sure the **run-it-twice** moment is visually clean: first run shows a card issued, second identical run shows a refusal with the provenance panel open, reading the record the first run itself wrote. Nothing scripted, no second fixture.
- **Stretch, if there's time:** one editable form where a judge can change any PO field and press issue themselves. Ten lines of UI, turns the whole claim into something they can test rather than take your word on.
- Budget meter, if there's still time after all of the above. Lowest priority, cut first.

20:00-21:00: deploy, verify, rehearse

- Push, deploy
- **Verify replay works on the deployed URL, from your phone, with the seeded history intact.** This is the single most likely thing to quietly break.
- Rehearse your half of the pitch, twice, timed. Practice narrating the provenance panel in five seconds.

The pitch line, if a judge reduces your checks to a database constraint

"This is a pre-issuance three-way match, the same PO, receipt and invoice reconciliation every ERP does, just moved from after the invoice arrives, where the remedy is a dispute or a clawback, to before the card

exists at all."

What you're explicitly not building

No LLM anywhere in `verify()`. That's the whole point, determinism is what makes replay mean anything. If someone suggests an agent judging the checks, that's design decision 2 being undone, say no.

No delegated budget trees, no dashboard-as-the-product pivot. Reasoning in `THE-IDEA.md`.